

LLM-Assisted Automatic Assertion Generation and Formal Validation for RTL Designs

1. Introduction

Formal verification is a critical component of hardware design validation. SystemVerilog Assertions (SVA) are widely used to specify correctness properties such as safety, liveness, and invariants. However, writing high-quality assertions is time-consuming and requires deep domain knowledge.

Recent advances in Large Language Models (LLMs) suggest that they can assist in automatically generating candidate assertions from RTL code. However, LLM-generated assertions may be incomplete, redundant, or incorrect.

This project proposes a structured pipeline that separates assertion generation (using LLMs) from formal validation (using formal verification tools).

2. Problem Statement

Design and implement a system that automatically generates candidate assertions using an LLM, converts them into syntactically correct SVA, validates them using formal verification tools, and analyzes their correctness and usefulness.

3. Objectives

- Generate candidate assertions from RTL designs.
- Classify assertions as safety, liveness, or functional invariants.
- Validate assertions using formal verification tools.
- Analyze correctness, redundancy, and usefulness.
- Optionally implement a refinement loop using counterexamples.

4. System Architecture

Stage 1: RTL Input (FIFO, Arbiter, FSM, Datapath)
Stage 2: LLM-based Assertion Generation
Stage 3: Assertion Formalization into SVA

Stage 4: Formal Verification using SymbiYosys
Stage 5: Analysis and Classification

5. Methodology

Prompt Engineering: Experiment with RTL-only and RTL+spec prompts.

Assertion Classification: Tag assertions as safety, liveness, or invariant.

Formal Validation: Use SymbiYosys for bounded model checking and counterexample generation.

Metrics: Measure precision, redundancy rate, and improvement after refinement.

6. Tools Used

- Verilog/SystemVerilog
- SymbiYosys
- Python scripting
- Large Language Models (ChatGPT/Claude)
- GTKWave (optional)

7. Expected Outcomes

- Automated assertion generation pipeline
- Verified SVA properties
- Identification of invalid or redundant assertions
- Quantitative evaluation of LLM effectiveness
- Insights into refinement using counterexamples

8. Deliverables

- Complete project report
- RTL source files
- Generated SVA files
- Prompt logs and LLM outputs
- Formal verification logs